



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

嵌入式系统设计技术

课设项目设计报告

学 期:	2025-2026 学年 第二学期
专 业:	计算机科学与技术 (实验班)
学号姓名:	23071005 侯景祺
项目名称:	MP3 播放器
完成日期:	2026.05.21

教师评语					
得 分		教师签字		评阅日期	

一、 功能设计

本项目设计的是一款基于 ESP32-C3 主控的触摸式 MP3 播放器。系统以 MP3 播放模块为核心音频输出单元，以 TFT LCD 触摸屏作为主要人机交互界面，同时结合 DS1307 实时时钟模块、EEPROM 记忆存储功能以及 Wi-Fi 物联网连接功能，实现本地触摸控制和手机 APP 远程控制相结合的音乐播放器。

本作品主要面向桌面、宿舍、教室、小型展示场景等近距离音频播放需求。用户既可以直接通过触摸屏完成播放、暂停、上一首、下一首、音量调节、播放模式切换、曲单选择等操作，也可以通过 Blinker APP 在手机端远程控制播放器。与普通按键式 MP3 播放器相比，本系统增加了图形化界面、歌曲列表、锁屏防误触、播放进度显示、本地时间显示以及 Wi-Fi 控制功能，使作品具有更好的交互性和完整性。

1. 音频播放控制功能

音频播放控制是本系统的核心功能。系统通过 WT2003H MP3 播放模块读取 Micro SD 卡中的 MP3 文件，并由 ESP32-C3 通过 UART 串口向 MP3 模块发送控制指令。用户可以通过 TFT 触摸屏或手机 APP 控制当前音频的播放状态。

正常播放模式下，播放器支持以下基本操作：

- 1) 播放当前选中的音频文件；
- 2) 暂停正在播放的音频；
- 3) 从暂停状态恢复播放；
- 4) 切换到上一首音频；
- 5) 切换到下一首音频；
- 6) 根据当前播放模式决定下一首的播放顺序。

当用户点击主界面中央的播放/暂停按钮时，系统会判断当前是否已经开始播放。如果还未开始播放，则直接播放当前曲目；如果已经开始播放，则向 MP3 模块发送暂停或继续播放指令。为了让界面状态和实际播放状态保持一致，系统设置了 `isPlaying` 和 `hasStarted` 两个状态变量。其中，`isPlaying` 用于表示当前是否处于播放状态，`hasStarted` 用于表示播放器是否已经启动过播放流程。这样可以避免第一次点击播放键时误执行暂停操作。

上一首和下一首功能根据当前曲目编号进行切换。当播放到第一首时点击上一首，系统会自动跳转到最后一首；当播放到最后一首时点击下一首，系统会自动回到第一首，从而形成完整的循环播放逻辑。

2. 播放模式切换功能

系统设计了三种播放模式，分别为列表循环、单曲循环和随机播放。用户可以通过主界面的 MODE 按键或 Blinker APP 中的模式按键进行切换。

三种播放模式的具体作用如下：

- 1) 列表循环模式

系统按照歌曲编号顺序播放。当播放到最后一首后，再次切换下一首时会回到第一首，适合普通连续听歌场景。

- 2) 单曲循环模式

系统重复播放当前音频文件，适合用户反复播放某一首音乐、语音素材或提示音的场景。

- 3) 随机播放模式

系统在所有歌曲中随机选择下一首进行播放，并尽量避免连续两次随机到同一首歌曲，提高播放的随机性和趣味性。

播放模式通过 `playMode` 变量进行记录，取值分别对应三种模式。每次点击 **MODE** 按键时，系统将 `playMode` 循环加一，并调用 MP3 模块对应的播放模式设置函数。界面中的模式文字也会同步刷新，使用户能够清楚看到当前处于 **Loop**、**Single** 还是 **Random** 模式。

3. TFT LCD 主界面显示功能

TFT LCD 是本系统最主要的信息显示和操作界面。系统采用图形化 UI 设计，将播放器信息分区显示，避免简单堆叠文字，提高界面的可读性和美观性。

主界面主要包括以下几个区域：

1) 顶部状态栏

显示作品名称和当前系统时间。时间由 **DS1307** 实时时钟模块提供，可显示小时、分钟和秒。

2) 当前歌曲信息卡片

显示当前播放歌曲名称、播放状态和唱片图案装饰。歌曲名称较长时会自动截断，防止文字超出屏幕范围。

3) 播放进度条

显示当前歌曲的估算播放进度。进度条下方显示当前已播放时间和歌曲总时长，使用户能够直观看到播放进程。

4) 播放状态信息栏

显示当前歌曲编号、歌曲总数、音量大小和播放模式。

5) 控制按键区域

提供上一首、播放/暂停、下一首、曲单、模式、锁屏、音量减和音量加等操作按键。

主界面采用深色背景、圆角卡片和高亮色按钮设计，使不同功能区域区分明显。中央播放按钮使用较大的尺寸显示，并根据播放状态改变颜色和图标：播放状态下显示暂停图标，暂停状态下显示播放三角图标，从而降低用户误操作的可能性。

4. 歌曲列表与选曲播放功能

为了方便用户查看和选择 **SD** 卡中的音频文件，系统设计了歌曲列表界面。用户在主界面点击 **LIST** 按键后进入曲单页面，可以浏览当前播放器中的歌曲名称，并选择指定歌曲播放。

歌曲列表界面的功能包括：

- 1) 分页显示歌曲名称；
- 2) 显示当前页码和总页数；
- 3) 高亮显示当前正在播放的歌曲；
- 4) 显示每首歌的编号和时长；
- 5) 支持上一页和下一页翻页；
- 6) 支持点击任意歌曲卡片后立即播放该歌曲；
- 7) 支持返回播放器主界面。

本系统每页显示 5 首歌曲，既能保证歌曲信息足够清晰，又能避免单页内容过于拥挤。每首歌曲以卡片形式显示，左侧为歌曲编号，中间为歌曲名称，右侧为歌曲时长。当前播放的歌曲会使用高亮边框和特殊颜色显示，方便用户快速识别当前播放项。

当用户点击某一首歌曲时，系统会计算触摸点所在的行号，再根据当前页码得到对应的歌曲编号，然后调用播放函数播放该歌曲，并自动返回主播放界面。这样用户不需要多次确认，操作过程更加直接。

5. 音量调节功能

系统支持通过触摸屏和手机 APP 两种方式调节音量。触摸屏主界面提供 V- 和 V+ 两个按键，分别用于降低和提高音量。手机端则可以通过音量加、音量减按钮，或音量滑条直接设置音量值。

音量范围设置为 0 至 31。每次调节音量时，系统都会使用 `constrain()` 函数限制音量范围，防止音量值超出 MP3 模块支持的范围。例如，当音量已经为 0 时继续点击 V-，系统仍会保持 0；当音量已经为 31 时继续点击 V+，系统仍会保持 31。

音量改变后，系统会立即向 MP3 模块发送新的音量值，并刷新 TFT 界面和 Blinker APP 显示内容，使本地界面和手机端状态保持一致。

6. 播放进度显示功能

为了提高播放器的信息完整性，系统增加了播放进度显示功能。由于 MP3 模块不一定能够实时返回精确播放进度，因此本系统采用软件计时的方式估算当前播放时间。

每次播放新歌曲时，系统会记录当前 `millis()` 时间作为歌曲开始时间。当歌曲暂停时，系统记录暂停时刻；当歌曲恢复播放时，将暂停持续的时间累计到暂停时间变量中。这样可以避免暂停期间进度条继续前进，使显示结果更符合实际播放状态。

播放进度条根据以下信息计算：

- 1) 当前歌曲的预设总时长；
- 2) 当前歌曲开始播放的时间；
- 3) 暂停累计时间；
- 4) 当前系统运行时间。

系统每隔约 500ms 更新一次进度条，不会频繁刷新整个屏幕，从而减少 TFT 屏幕闪烁。进度条下方同时显示已播放时间和歌曲总时长，使用户能够直观看到当前播放进程。

7. 本地时间显示与时间设置功能

系统使用 DS1307 实时时钟芯片为播放器提供本地时间。正常播放界面顶部会实时显示当前时间，使播放器不仅具有音乐播放功能，也具有电子时钟功能。

当用户点击主界面右上角的时间区域时，系统进入时间设置界面。时间设置界面提供 H+、M+、S+、H-、M-、S- 六个按键，分别用于调整小时、分钟和秒。用户调整完成后点击 SAVE 按键，系统会将新的时间写入 DS1307 芯片；如果不想保存，则点击 BACK 返回主界面。

时间设置功能的设计使系统不依赖电脑或串口工具也能完成时间校准，增强了作品的独立使用能力。

8. 掉电记忆功能

系统具有上次播放记忆功能。当用户播放某一首歌曲时，系统会将当前歌曲编号写入 EEPROM。下次系统重新上电后，会自动读取 EEPROM 中保存的曲目编号，并将该歌曲作为开机后的当前待播放音频。

该功能可以避免每次开机都从第一首歌曲开始，提高用户体验。例如，用户上一次播放到第 4 首歌曲，设备断电后再次启动，系统会自动恢复到第 4 首作为当前歌曲。若 EEPROM 中读取到的歌曲编号

无效，例如编号小于 1 或大于歌曲总数，系统会自动恢复为第 1 首，保证程序不会因为异常数据而出错。

9. 锁屏与防误触功能

为了防止播放器在携带或放置过程中被误触，系统设计了锁屏功能。用户可以通过主界面的 LOCK 按键主动进入锁屏模式，也可以在一定时间无操作后自动进入锁屏模式。

锁屏模式下，TFT 屏幕会显示专门设计的锁屏界面，包括当前时间、锁形图案、装饰圆环和提示文字。锁屏后，系统不会响应播放、暂停、切歌、调音量等普通触摸操作，触摸屏幕只会执行解锁操作。这样可以避免误触导致歌曲切换、音量变化或播放状态改变。

自动锁屏功能通过 `lastUserAction` 变量记录用户最后一次操作时间，并在主循环中持续判断当前时间与最后操作时间的差值。当无操作时间超过设定阈值后，系统自动调用锁屏函数进入锁屏页面。解锁后，系统会返回锁屏前所在的页面，例如原来在歌曲列表界面，解锁后仍回到歌曲列表界面。

10. Wi-Fi 与 Blinker APP 远程控制功能

为了满足手机端控制要求，系统接入 Blinker 物联网 APP。ESP32-C3 通过 Wi-Fi 连接网络，并使用 Blinker 设备密钥与手机 APP 建立通信。用户可以在手机端查看当前歌曲名称、播放状态、播放模式、系统时间、当前曲目编号和音量值，也可以远程控制播放器。

Blinker APP 端主要支持以下功能：

- 1) 播放和暂停当前音频；
- 2) 切换上一首；
- 3) 切换下一首；
- 4) 切换播放模式；
- 5) 调整音量；
- 6) 查看当前歌曲名称；
- 7) 查看当前播放状态；
- 8) 查看当前系统时间；
- 9) 远程进入或退出锁屏状态；
- 10) 切换播放器界面和曲单界面。

本地触摸屏和手机 APP 使用同一套控制函数。例如，触摸屏点击下一首和 APP 点击下一首按钮，最终都会调用 `playNext()` 函数。这样可以避免本地控制和远程控制逻辑不一致的问题。每次 APP 控制完成后，系统都会刷新 TFT 屏幕和 APP 显示数据，使两端信息保持同步。

二、 硬件电路设计

本系统以 ESP32-C3 开发板作为主控核心，主要由 TFT 触摸屏模块、MP3 播放模块、RTC&EEPROM 模块、microSD 卡、扬声器和手机 APP 端组成。各模块之间通过不同通信接口连接，由 ESP32-C3 统一完成控制和数据处理。系统硬件结构如图 1 所示。

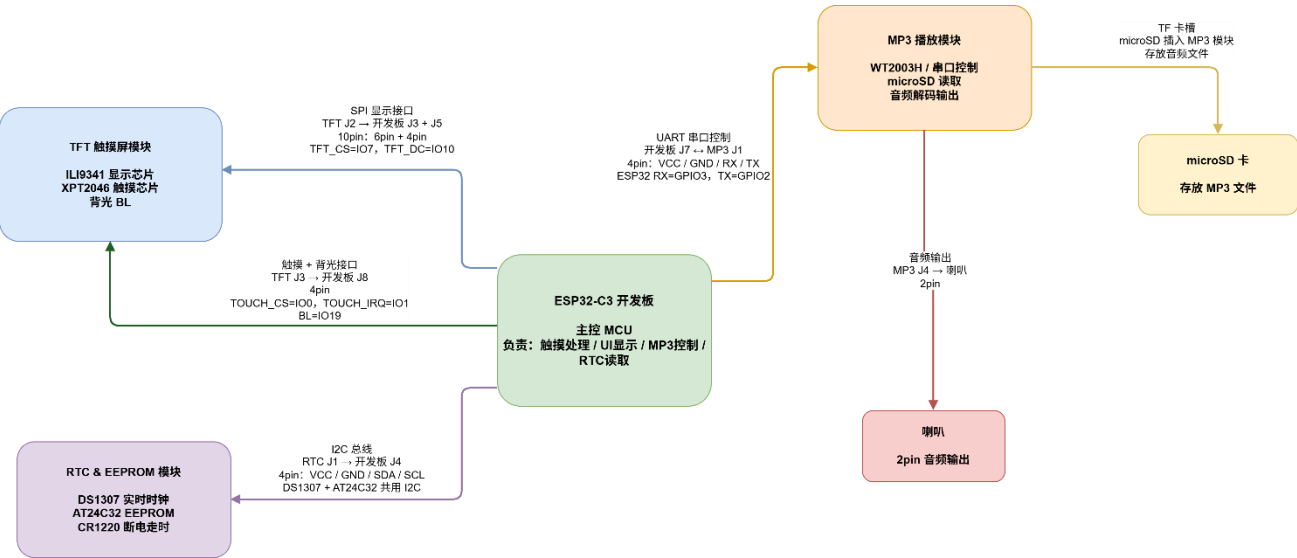


图 1 硬件电路示意图

1. ESP32-C3 主控模块

ESP32-C3 是本系统的核心控制模块，负责协调各个硬件模块工作。它主要完成触摸输入识别、TFT 屏幕界面刷新、MP3 播放控制、RTC 时间读取、EEPROM 数据保存以及 Wi-Fi 通信等任务。

当用户点击屏幕上的播放、暂停、上一首、下一首、音量调节等按钮时，ESP32-C3 会先读取触摸坐标，然后判断用户点击的是哪个功能区域，再向 MP3 播放模块发送对应的控制指令。同时，ESP32-C3 也会刷新屏幕上的歌曲名称、播放状态、音量和播放进度等信息。

2. TFT 触摸屏模块

TFT 触摸屏模块用于显示播放器界面和接收用户操作。该模块包含 ILI9341 显示芯片和 XPT2046 触摸芯片，可以同时实现彩色界面显示和触摸控制。

在本系统中，TFT 屏幕主要显示当前歌曲名称、播放状态、播放进度、音量、播放模式、系统时间和歌曲列表等内容。用户可以直接点击屏幕完成播放、暂停、切歌、调节音量、进入曲单、切换模式和锁屏等操作。

TFT 显示部分通过 SPI 接口与 ESP32-C3 通信，其中 TFT_CS 接 IO7，TFT_DC 接 IO10。触摸部分中，TOUCH_CS 接 IO0，TOUCH_IRQ 接 IO1。屏幕背光 BL 接 IO19，用于控制屏幕背光。

3. MP3 播放模块

MP3 播放模块用于读取 microSD 卡中的 MP3 文件，并完成音频解码和播放。ESP32-C3 不直接处理音频数据，只通过 UART 串口向 MP3 模块发送控制命令。

本系统中，ESP32-C3 通过 IO2 和 IO3 与 MP3 模块通信。其中 IO2 作为 ESP32-C3 的 TX 引脚，用于向 MP3 模块发送指令；IO3 作为 RX 引脚，用于接收 MP3 模块返回的信息。

用户点击播放、暂停、上一首、下一首、音量调节等按钮后，ESP32-C3 会通过串口控制 MP3 模块完成对应操作。这样可以减少主控芯片的负担，使系统结构更加清晰。

4. microSD 卡与扬声器

microSD 卡用于存放 MP3 音频文件，插入 MP3 播放模块的卡槽中。MP3 模块会从 microSD 卡中读取音频文件，并根据 ESP32-C3 发送的歌曲编号播放对应音乐。

扬声器连接在 MP3 播放模块的音频输出接口上，用于输出声音。音频解码和声音输出都由 MP3 播放模块完成，ESP32-C3 只负责发送控制指令。

5. RTC&EEPROM 模块

RTC&EEPROM 模块用于提供实时时间和保存播放记录。该模块中包含 DS1307 实时时钟芯片和 AT24C32 EEPROM 存储芯片，两者都通过 I2C 总线与 ESP32-C3 通信。

DS1307 用于提供当前时间，系统可以在主界面和锁屏界面显示小时、分钟和秒。用户也可以通过时间设置界面对时间进行修改。

AT24C32 EEPROM 用于保存上次播放的歌曲编号。当系统重新上电时，ESP32-C3 会读取 EEPROM 中保存的数据，并恢复到上一次播放的歌曲，从而实现掉电记忆功能。

该模块通过 VCC、GND、SDA、SCL 四根线与 ESP32-C3 连接，使用 I2C 接口进行通信。

6. Wi-Fi 与手机 APP 通信

ESP32-C3 内部带有 Wi-Fi 功能，因此本系统不需要额外增加 Wi-Fi 模块。系统通过 Wi-Fi 连接 Blinker APP，实现手机端远程控制。

用户可以在 Blinker APP 中控制播放、暂停、上一首、下一首、音量调节、播放模式切换和锁屏等功能。同时，APP 端也可以显示当前歌曲名称、播放状态、音量和系统时间等信息。

手机 APP 控制和本地触摸屏控制使用的是同一套播放控制函数，因此两种控制方式不会互相冲突。

7. 硬件接口分配

本系统各模块接口分配如下：

- 1) TFT 显示模块使用 SPI 接口，用于传输屏幕显示数据；
- 2) XPT2046 触摸芯片使用触摸片选和中断引脚，用于检测屏幕触摸；
- 3) MP3 播放模块使用 UART 串口，用于接收播放控制指令；
- 4) RTC&EEPROM 模块使用 I2C 总线，用于读取时间和保存数据；
- 5) Wi-Fi 功能由 ESP32-C3 内部完成，用于连接 Blinker APP。

这种设计使各模块分工明确，通信接口之间互不冲突，能够保证系统稳定运行。

三、 程序代码设计

本系统程序基于 Arduino 框架进行开发，主要完成 MP3 播放控制、TFT 触摸屏界面显示、RTC 时间读取、EEPROM 掉电记忆、锁屏防误触以及 Blinky APP 远程控制等功能。程序采用模块化设计方式，将不同功能分别封装成对应函数，便于后续修改和扩展。

程序中主要使用的函数库如下：

```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#include <XPT2046_Touchscreen.h>
#include <RTCLib.h>
#include "WT2003S_Player.h"
```

其中，SPI.h 用于 TFT 屏幕和触摸芯片通信，Wire.h 用于 I2C 通信，Adafruit_ILI9341.h 用于控制 TFT 彩色屏幕，XPT2046_Touchscreen.h 用于读取触摸屏坐标，RTCLib.h 用于读取 DS1307 实时时钟，WT2003S_Player.h 用于控制 MP3 播放模块。

程序中还定义了多个重要状态变量，例如：

```
int currentTrack = 1;
bool isPlaying = false;
int volume = 15;
int playMode = 0;
bool isScreenLocked = false;
```

其中，currentTrack 表示当前歌曲编号，isPlaying 表示是否正在播放，volume 表示当前音量，playMode 表示播放模式，isScreenLocked 表示是否处于锁屏状态。

1. 程序总体结构设计

程序整体由 setup() 和 loop() 两部分组成。setup() 函数主要负责系统初始化，包括初始化串口、TFT 屏幕、触摸屏、RTC 模块、MP3 播放模块、歌曲信息和 Blinky APP 连接等。初始化完成后，系统进入播放器主界面。

```
void setup() {
    Serial.begin(115200);

    Wire.begin();

    tft.begin();
    tft.setRotation(1);

    ts.begin();
    ts.setRotation(1);

    initRTC();

    mp3Serial.begin(9600, SERIAL_8N1, MP3_RX_PIN, MP3_TX_PIN);
```

```

    Mp3Player.init(mp3Serial);

    loadSongNames();

    setVolumeSafe(volume);
    playTrack(currentTrack, false);

    drawPlayScreen();

    setupBlinkerApp();
}

```

`loop()` 函数是程序的主循环，主要负责持续检测触摸操作、处理 **Blinker APP** 指令、更新时间显示、刷新播放进度条以及判断是否需要自动锁屏。

```

void loop() {
    Blinker.run();

    handleTouch();

    if (isScreenLocked) {
        updateLockScreenClock(false);
        return;
    }

    if (millis() - lastUserAction > SCREEN_AUTO_LOCK_TIMEOUT_MS) {
        enterLockScreen();
        return;
    }

    drawTopStatus(false);
    updateProgressTask();
}

```

通过这种结构，系统可以持续运行并实时响应用户操作。

2. MP3 播放控制模块设计

MP3 播放控制模块是本系统的核心部分，主要负责播放、暂停、上一首、下一首、音量调节和播放模式切换。ESP32-C3 通过 UART 串口向 MP3 模块发送控制指令，MP3 模块负责读取 microSD 卡中的音频文件并输出声音。

播放指定歌曲的函数如下：

```

void playTrack(int trackNum, bool save) {
    currentTrack = constrain(trackNum, 1, TOTAL_SONGS);

    Mp3Player.volume(volume);
    Mp3Player.playSDRootSong(mp3ArgFromTrack(currentTrack));
}

```

```

    resetTrackProgress();

    isPlaying = true;
    hasStarted = true;

    if (save) {
        saveMemory();
    }

    updateBlinkerInfo();
}

```

该函数首先限制歌曲编号范围，防止编号超出歌曲总数；然后设置当前音量，并调用 **MP3** 播放模块函数播放指定曲目。播放新歌曲后，程序会重置播放进度，同时将播放状态设置为正在播放。

播放/暂停功能通过 **togglePlay()** 函数实现：

```

void togglePlay() {
    if (!hasStarted) {
        playTrack(currentTrack);
        return;
    }

    Mp3Player.pause_or_play();

    if (isPlaying) {
        pausedAtMillis = millis();
        isPlaying = false;
    } else {
        pausedAccumMillis += millis() - pausedAtMillis;
        isPlaying = true;
    }

    updateBlinkerInfo();
    drawProgressBar(true);
}

```

该函数会根据当前播放状态切换播放或暂停。如果系统还没有开始播放，则第一次点击会直接播放当前歌曲；如果已经在播放，则执行暂停；如果处于暂停状态，则恢复播放。

上一首和下一首功能通过 **playPrev()** 和 **playNext()** 实现：

```

void playNext() {
    if (playMode == 2) {
        currentTrack = random(1, TOTAL_SONGS + 1);
    } else {
        currentTrack = (currentTrack >= TOTAL_SONGS) ? 1 : currentTrack + 1;
    }
}

```

```
    playTrack(currentTrack);  
}
```

当播放模式为随机播放时，下一首会随机选择歌曲；否则按照歌曲编号顺序播放。这样可以实现列表循环和随机播放两种逻辑。

3. TFT 显示与触摸控制模块设计

TFT 显示与触摸控制模块负责人机交互。程序中通过 `drawPlayScreen()` 绘制播放器主界面，通过 `drawListScreen()` 绘制歌曲列表界面，通过 `drawTimeSetScreen()` 绘制时间设置界面。

播放器主界面主要显示歌曲名称、播放状态、进度条、音量、播放模式和控制按钮。界面绘制函数如下：

```
void drawPlayScreen() {  
    tft.fillScreen(UI_BG);  
    drawHeader("Pocket MP3");  
  
    tft.fillRoundRect(12, 43, 296, 92, 16, UI_CARD);  
    tft.drawRoundRect(12, 43, 296, 92, 16, UI_LINE);  
  
    drawButton(28, 154, 66, 38, "<<", tft.color565(50, 58, 82));  
    drawButton(226, 154, 66, 38, ">>", tft.color565(50, 58, 82));  
  
    drawButton(16, 203, 58, 29, "LIST", UI_ACCENT);  
    drawButton(82, 203, 64, 29, "MODE", UI_PURPLE);  
    drawButton(154, 203, 64, 29, "LOCK", tft.color565(245, 145, 58));  
  
    updateDynamicUI();  
}
```

触摸控制主要由 `handleTouch()` 函数完成。程序先读取触摸坐标，再根据当前页面状态判断用户点击的是哪个功能区域。

```
void handleTouch() {  
    if (!ts.touched()) return;  
  
    TS_Point p = ts.getPoint();  
  
    int x = map(p.x, 300, 3800, 0, 320);  
    int y = map(p.y, 3800, 200, 0, 240);  
  
    if (currentPage == PAGE_PLAYER) {  
        if (x >= 120 && x <= 200 && y >= 148 && y <= 198) {  
            togglePlay();  
        } else if (x >= 28 && x <= 94 && y >= 154 && y <= 192) {  
            playPrev();  
        } else if (x >= 226 && x <= 292 && y >= 154 && y <= 192) {  
            playNext();  
        }  
    }  
}
```

```
    }  
    }  
}
```

通过这种方式，屏幕上不同区域对应不同操作，用户可以直接点击屏幕完成播放控制。

4. 页面状态机模块设计

为了避免不同界面的触摸操作发生冲突，程序中设计了页面状态机。系统使用 `PageState` 枚举变量表示当前所处页面。

```
enum PageState {  
    PAGE_PLAYER,  
    PAGE_LIST,  
    PAGE_TIME_SET  
};
```

```
PageState currentPage = PAGE_PLAYER;
```

其中，`PAGE_PLAYER` 表示播放主界面，`PAGE_LIST` 表示歌曲列表界面，`PAGE_TIME_SET` 表示时间设置界面。程序会根据当前页面状态执行不同的触摸处理逻辑。

例如，进入歌曲列表界面时，程序会修改当前页面状态：

```
void enterListPage() {  
    currentPage = PAGE_LIST;  
    listPage = (currentTrack - 1) / SONGS_PER_PAGE;  
    drawListScreen();  
}
```

返回播放主界面时，程序会执行：

```
void enterPlayerPage() {  
    currentPage = PAGE_PLAYER;  
    drawPlayScreen();  
}
```

状态机的作用是让程序清楚地区分当前处于哪个界面。例如，在播放界面点击某一区域可能是“下一首”，而在歌曲列表界面点击相同位置可能是“选择歌曲”。通过状态机管理，可以避免误判触摸操作，使系统逻辑更加清晰。

5. 锁屏与 APP 控制模块设计

系统设计了锁屏功能，防止用户误触屏幕。锁屏状态由 `isScreenLocked` 变量表示。

```
bool isScreenLocked = false;
```

用户可以点击主界面的 `LOCK` 按键进入锁屏，也可以在一段时间无操作后自动锁屏。进入锁屏时，程序调用 `enterLockScreen()` 函数：

```
void enterLockScreen() {  
    if (isScreenLocked) return;  
  
    isScreenLocked = true;  
    lastLockClockUpdate = 0;
```

```
drawLockScreen();  
}
```

锁屏界面会显示当前时间、锁形图案和提示文字。锁屏后，普通播放控制不会被触发，用户点击屏幕只会执行解锁操作。

```
void exitLockScreen() {  
    isScreenLocked = false;  
    lastUserAction = millis();  
  
    if (currentPage == PAGE_PLAYER) {  
        drawPlayScreen();  
    } else if (currentPage == PAGE_LIST) {  
        drawListScreen();  
    } else if (currentPage == PAGE_TIME_SET) {  
        drawTimeSetScreen();  
    }  
}
```

此外，程序还接入了 **Blinker APP** 控制功能。**Blinker** 按钮回调函数可以实现手机端控制播放、暂停、上一首、下一首、音量调节和锁屏等功能。

例如，APP 端播放/暂停按钮对应函数如下：

```
void blinkerPlayCallback(const String & state) {  
    lastUserAction = millis();  
    togglePlay();  
    redrawAfterAppAction();  
}
```

这样，用户既可以通过 **TFT** 触摸屏控制播放器，也可以通过手机 **APP** 远程控制播放器。两种控制方式调用的是同一套播放函数，因此系统状态能够保持一致。

四、 测试

为验证本系统各功能是否能够正常运行，分别对 MP3 播放控制、TFT 触摸界面、歌曲列表、锁屏功能以及 Blinker APP 远程控制功能进行了测试。测试过程中观察屏幕显示、扬声器输出声音以及手机 APP 控制响应情况，测试结果如下表所示。

表 1 系统功能测试结果

测试项目	测试方法	预期结果	实际结果
MP3 播放测试	系统上电后点击主界面播放按钮	扬声器播放当前歌曲，屏幕显示播放状态	能正常播放歌曲，播放状态显示正确
播放控制测试	依次点击暂停、上一首、下一首按钮	音乐可暂停/继续播放，歌曲可正常切换	暂停、继续、上一首、下一首功能均正常
歌曲列表测试	点击 LIST 进入曲单界面，并选择其中一首歌曲	屏幕显示歌曲列表，点击歌曲后播放对应音频	曲单显示正常，点击歌曲后可切换播放
锁屏功能测试	点击 LOCK 按键或等待一段时间无操作	系统进入锁屏界面，触摸普通区域不会误操作	可正常进入锁屏，点击屏幕后可解锁
Blinker APP 控制测试	在手机 APP 中点击播放、暂停、切歌、音量调节按钮	播放器能响应 APP 指令，屏幕状态同步更新	APP 可正常控制播放器，界面信息同步正常

1. MP3 播放功能测试

首先将存有 MP3 文件的 microSD 卡插入 MP3 播放模块，然后给系统上电。系统启动后进入播放器主界面，点击播放按钮，观察扬声器是否有声音输出，同时查看屏幕上的歌曲名称和播放状态是否发生变化。

测试结果表明，系统可以正常读取并播放 microSD 卡中的 MP3 文件。播放开始后，TFT 屏幕能够显示当前歌曲名称、歌曲编号、播放状态和播放进度，说明 MP3 播放模块和 ESP32-C3 之间的串口通信正常。

2. 播放控制功能测试

在歌曲播放过程中，分别测试播放/暂停、上一首、下一首和播放模式切换功能。点击播放/暂停按钮后，音乐能够暂停，再次点击后可以继续播放。点击上一首或下一首按钮后，系统能够切换到对应歌曲，并且屏幕上的歌曲名称和歌曲编号同步更新。

测试结果表明，播放控制功能基本正常，触摸屏操作和 MP3 模块响应一致，没有出现按钮无效或界面不同步的问题。



图 2 播放菜单示意图

3. 歌曲列表功能测试

点击主界面中的 LIST 按键进入歌曲列表界面。进入后观察歌曲名称、歌曲编号、歌曲时长和当前播放歌曲高亮状态是否正确。随后点击列表中的某一首歌曲，测试系统是否能够播放被选中的音频。

测试结果表明，歌曲列表界面能够正常显示多首歌曲，当前播放歌曲可以高亮显示。点击指定歌曲后，系统能够播放对应曲目，并自动返回播放主界面，说明曲单选择功能正常。



图 3 曲目列表示意图

4. 锁屏功能测试

在播放器主界面点击 LOCK 按键，观察系统是否进入锁屏界面。进入锁屏后，屏幕显示锁屏图案、当前时间和提示文字。此时点击屏幕，系统执行解锁操作并返回原来的播放器界面。

随后测试自动锁屏功能，在一段时间内不进行任何触摸操作，观察系统是否会自动进入锁屏界面。测试结果表明，手动锁屏和自动锁屏功能均能正常工作，锁屏后可以防止误触，提高了系统使用的稳定性。

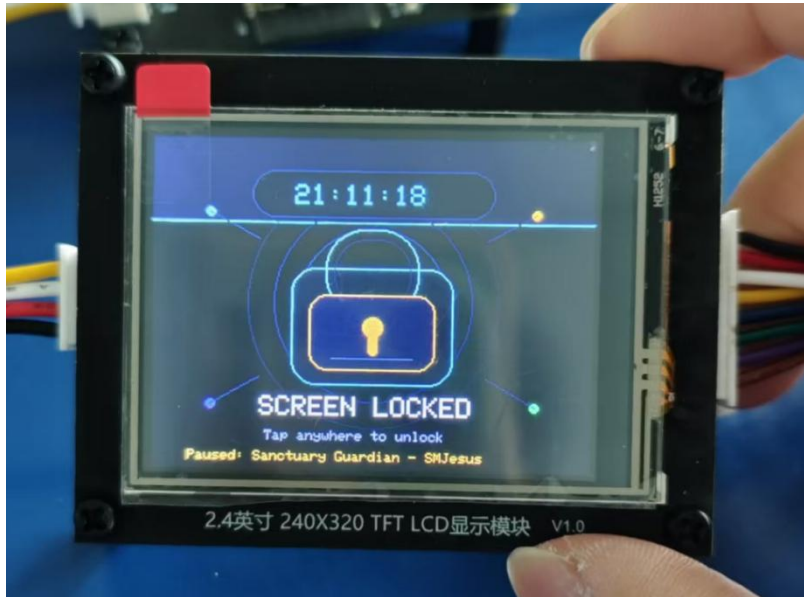


图 4 锁屏示意图

5. Blinker APP 控制功能测试

打开手机端 Blinker APP，连接对应设备后，分别点击播放/暂停、上一首、下一首、音量调节和模式切换等控件。观察播放器是否能够响应手机端指令，同时查看 TFT 屏幕上的歌曲状态、音量和播放模式是否同步更新。

测试结果表明，ESP32-C3 可以正常连接 Wi-Fi 并接收 Blinker APP 指令。手机 APP 控制播放器时，系统能够正确执行对应操作，说明远程控制功能正常。



图 5 APP 控制示意图

五、 小结

本次 MP3 播放器课程设计过程中，整体功能基本按照预期完成，实现了本地触摸控制、歌曲播放、曲单显示、锁屏防误触以及 Blinky APP 远程控制等功能。但在调试过程中也遇到了一些问题，其中最主要的问题是程序代码量较大，初次编译和上传时出现失败。经过排查发现，原因是没有正确选择 ESP32-C3 的代码分区模式，默认分区下可用程序存储空间较小，无法容纳包含 TFT 图形界面、触摸控制、RTC、MP3 控制和 Wi-Fi/Blinky 功能在内的完整程序。后续将分区方案调整为 Huge APP 后，程序才可以正常编译并上传。

通过这个问题可以看出，嵌入式系统开发不仅要关注功能能否实现，还必须考虑硬件资源限制。单片机的 Flash、RAM、接口数量和处理能力都比较有限，如果程序设计过于复杂，就可能出现存储空间不足、运行不稳定或响应速度下降等问题。这也让我联想到硬件综合课程设计中在边缘设备上部署 CV 识别器的经历。当时设备只有双核 ARM A9 处理器和约 500MB 内存，计算能力和存储资源都非常有限，因此必须通过模型裁剪、降低输入分辨率、减少无用模块、优化推理流程等方式尽量降低资源占用。相比电脑端开发，嵌入式和边缘设备开发更强调在有限资源下实现尽可能稳定、完整的功能。

经过本次项目实践，我进一步理解了模块化设计、状态机管理、外设通信和资源优化的重要性。后续如果继续改进本作品，可以进一步压缩 UI 代码、减少重复绘图、优化 APP 通信逻辑，并增加播放统计、收藏歌曲等扩展功能，使系统在功能完整性和资源占用之间取得更好的平衡。